

CTE412 · INTEGRATED SYSTEMS TECHNOLOGY

Docker Step by Step

Follow along, one step at a time

Unit IV · Advanced CI/CD Pipeline Practices

Module	CTE412 Integrated Systems Technology
Programme	BE in Information Technology
Unit	Unit IV — Advanced CI/CD Pipeline Practices
Topic	Containerisation with Docker (small images throughout)
Document type	Step-by-step student guide
Prerequisites	None. Start from zero.

Before you start

This guide assumes you know nothing about Docker. Every command is written out for you, and after each one you are shown what should appear on your screen so you can check your own result before moving on.

There are 26 numbered steps. Do them in order, at a computer, typing each command yourself instead of copying and pasting — typing is how the commands become familiar. If a check fails, stop and fix it before continuing, because later steps build on earlier ones.

Contents

How to Use This Guide	3
Reading the commands.....	3
The three kinds of boxes	3
About the images we use	3
What you need.....	4
Part A: Getting Ready.....	4
Part B: Your First Containers	5
Part C: Running a Real Program	7
Part D: Looking Inside a Container	10
Part E: Building Your Own Image	12
Part F: Keeping Data Safe	16
Part G: Connecting Containers.....	17
Part H: Running Several Containers Together.....	18
Part I: Cleaning Up	20
Quick Reference	21
If Something Goes Wrong	22
Check What You Have Learned	23

How to Use This Guide

This guide takes you from knowing nothing about Docker to running your own multi-container application. Follow it in order and type every command yourself.

Reading the commands

Commands are shown in shaded boxes like this one:

TYPE THIS

```
$ docker --version
```

The **\$** sign marks where a command starts. **Do not type it.** Type only what comes after it.

After most commands you will see what Docker prints back:

YOU SHOULD SEE

```
Docker version 27.3.1, build ce12230
```

Your numbers may be different. That is normal. What matters is that the *shape* of the output matches.

The three kinds of boxes

Box	What it means
Type this	A command for you to type and run.
You should see	What Docker prints back. Compare it with your screen.
File	Text to save into a file using your editor. Do not type it into the terminal.

CHECK YOUR WORK

Green boxes like this one tell you how to confirm a step worked before you move on. Never skip them. If a check fails, fix it before continuing — later steps depend on earlier ones.

About the images we use

Docker downloads *images* from the internet. Some images are very large. Throughout this guide we use the smallest version of every image, so you download as little as possible.

Most images offer a variant built on **Alpine Linux**, a very small version of Linux. You ask for it by adding **-alpine** or **:alpine** to the image name.

Image	Normal size	Small version we use	Size
nginx	192 MB	nginx:alpine	52 MB
python	1.02 GB	python:3.12-alpine	52 MB
redis	117 MB	redis:7-alpine	41 MB

Image	Normal size	Small version we use	Size
httpd	148 MB	<code>httpd:alpine</code>	56 MB
ubuntu / alpine	78 MB	<code>alpine</code>	8 MB

Everything in this guide together downloads about **210 MB**. Using the normal versions would download over **1.5 GB** — seven times more.

TIP

This matters beyond saving time. Smaller images start faster, use less disk space, and contain fewer programs that could contain security problems. Professionals choose small images for exactly these reasons.

What you need

- Docker Desktop installed and running, **or** a free browser playground.
- A text editor. **Visual Studio Code** is recommended.
- About two hours.

Part A: Getting Ready

1 Open a terminal

Windows: press the Start button, type `powershell`, and open **Windows PowerShell**.

macOS: press Command and Space, type `terminal`, and press Enter.

Linux: press Ctrl, Alt and T together.

A window opens with a blinking cursor. This is where you type commands.

2 Check that Docker is installed

Type this

```
$ docker --version
```

You should see

```
Docker version 27.3.1, build ce12230
```

This tells you the Docker *command* is installed. But the command alone cannot do anything — it needs a background service called the **daemon** to do the real work. Check that too.

Type this

```
$ docker info
```

You should see

```
Client:
  Version:      27.3.1

Server:
  Containers: 0
  Images: 0
  Server Version: 27.3.1
```

CHECK YOUR WORK

The output must contain a section beginning with **Server:**. If it does, Docker is fully working and you can continue.

WARNING

If you see **Cannot connect to the Docker daemon**, the background service is not running. Open **Docker Desktop** from your Start menu or Applications folder and wait until the whale icon stops moving. Then run **docker info** again.

3 Make a folder for your work

You will create several files during this guide. Keep them together in one place.

Type this

```
$ mkdir docker-lab
$ cd docker-lab
```

mkdir makes a new folder. **cd** moves you into it. Everything you do from now on happens inside this folder.

Type this

```
$ pwd
```

This prints where you currently are. The path should end with **docker-lab**. On Windows PowerShell you can also use **pwd**.

Part B: Your First Containers

Before you run anything, two words. An **image** is a file containing a program and everything it needs. A **container** is that image actually running. One image can start many containers, the same way one program on your computer can be opened many times.

4 Run your very first container

hello-world is a tiny image made only for testing. It is 13 kilobytes — smaller than a photograph.

Type this

```
$ docker run hello-world
```

You should see

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

CHECK YOUR WORK

You must see the words **Hello from Docker!** on your screen. If you do, Docker downloaded an image from the internet and ran it successfully.

What just happened

1. You asked Docker to run an image called `hello-world`.
2. Docker looked on your computer. The image was not there.
3. Docker downloaded it from the internet.
4. Docker started a container from that image.
5. The container printed a message.
6. The message was the only job, so the container **stopped**.

5 Run it again and notice the difference

Type this

```
$ docker run hello-world
```

This time there is **no download**. The lines about pulling are gone and the message appears immediately.

The image is now saved on your computer. Docker only downloads an image the first time. Every run after that is instant.

NOTE

This is why the registry mirror in your lab helps so much. Once one student downloads an image, it is stored nearby and everybody else gets it without using the internet at all.

6 See what is running

Type this

```
$ docker ps
```

You should see

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES

The list is empty. Nothing is running, because `hello-world` finished its job and stopped.

To see stopped containers as well, add `-a`, which means *all*.

Type this

```
$ docker ps -a
```

You should see

CONTAINER ID	IMAGE	STATUS	NAMES
9f2b1c4d8e7a	hello-world	Exited (0) 2 minutes ago	quirky_bell
3a7e5f1b9c2d	hello-world	Exited (0) 5 minutes ago	nifty_curie

Both containers you ran are still there, stopped. **Exited (0)** means it finished with no errors.

NOTE

Docker invented the names **quirky_bell** and **nifty_curie**. When you do not choose a name, Docker makes up a random one. Yours will be different. Later you will choose your own names.

7 Look at your images

Type this

```
$ docker images
```

You should see

REPOSITORY	TAG	IMAGE ID	SIZE
hello-world	latest	d2c94e258dcb	13.3kB

One image, and it is tiny. Now download a small Linux image so you can compare.

Type this

```
$ docker pull alpine
```

pull downloads an image without running it. Alpine is a complete Linux system in about 8 megabytes.

Type this

```
$ docker images
```

You should see

REPOSITORY	TAG	IMAGE ID	SIZE
alpine	latest	a606584aa9aa	7.8MB
hello-world	latest	d2c94e258dcb	13.3kB

CHECK YOUR WORK

You should now have two images listed. Note how small Alpine is — a full Linux system in less space than a short video.

Part C: Running a Real Program

hello-world prints a message and stops. Real programs — web servers, databases — need to keep running. This part uses **Nginx**, a web server.

8 Start a web server

Type this

```
$ docker run -d -p 8080:80 --name web nginx:alpine
```

Docker downloads the image (52 MB) and prints a long string of letters and numbers. That is the container's ID.

There are four new things in that command:

Part	What it does
<code>-d</code>	Run in the background so you get your terminal back. Without this, the terminal is stuck until the server stops.
<code>-p 8080:80</code>	Connect port 8080 on your computer to port 80 inside the container . Always in that order.
<code>--name web</code>	Call this container <code>web</code> instead of a random name.
<code>nginx:alpine</code>	The image to use. The <code>:alpine</code> part asks for the small version.

Type this

```
$ docker ps
```

You should see

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
c3f8a91b2d47	nginx:alpine	Up 30 seconds	0.0.0.0:8080->80/tcp	web

This time the list is **not** empty. The status says **Up**, meaning it is still running.

9 Open it in your browser

Open a web browser and go to this address:

ADDRESS BAR

```
http://localhost:8080
```

CHECK YOUR WORK

You should see a page that says **Welcome to nginx!** You are looking at a web server running inside a container on your own computer.

WARNING

If the page does not load, check `docker ps` shows the container as **Up**. If you saw an error saying **port is already allocated**, something else on your computer is using port 8080. Remove the container with `docker rm -f web` and try again using `-p 8081:80`, then visit `http://localhost:8081`.

Understanding the two port numbers

This is the part beginners find hardest, so read it slowly.

A container is sealed off from your computer. Nginx is listening on port 80 **inside** the container, but nothing outside can reach it. The `-p` option opens a door.

HOW -P 8080:80 WORKS

Your computer	Container
-----	-----
localhost:8080	-----> port 80 (nginx)
^	^
you choose this	the program decides this

The **left** number is yours to choose. You could use 3000, 8085, or 9999. The **right** number is fixed by the program inside, so you cannot change it.

TIP

Remember it this way: the number on the **left** is the one you type into the browser.

10 Read the container's messages

Anything the program prints is saved by Docker. Read it with **logs**.

Type this

```
$ docker logs web
```

You should see

```
/docker-entrypoint.sh: Configuration complete; ready for start up
172.17.0.1 - - "GET / HTTP/1.1" 200 615
```

That last line is your browser visit being recorded. Refresh the browser page, then run **docker logs web** again — a new line appears.

To watch messages arrive live, add **-f**, which means *follow*.

Type this

```
$ docker logs -f web

# Refresh your browser and watch new lines appear.
# Press Ctrl and C together to stop watching.
```

NOTE

Pressing Ctrl+C only stops you *watching*. The container keeps running. Check with **docker ps** if you want to be sure.

11 Stop, start, and remove

A container can be stopped and started again, like pausing a program.

Type this

```
$ docker stop web
$ docker ps
```

The list is empty — it is no longer running. But it still exists:

Type this

```
$ docker ps -a
```

You should see

CONTAINER ID	IMAGE	STATUS	NAMES
c3f8a91b2d47	nginx:alpine	Exited (0) 5 seconds ago	web

Start it again. It keeps the same name and the same settings.

Type this

```
$ docker start web
$ docker ps
```

Refresh your browser — the page works again.

Stopping is not the same as removing. To delete a container completely, stop it first, then remove it.

Type this

```
$ docker stop web
$ docker rm web
```

TIP

You can do both at once with `docker rm -f web`. The `-f` means *force*: stop it and delete it in one step.

WARNING

If you ever see **Conflict. The container name "web" is already in use**, it means a container with that name still exists, probably stopped. Remove it with `docker rm -f web` and try again. This is the most common error in the whole guide.

Part D: Looking Inside a Container

A container holds a small Linux system. You can open a terminal inside it and look around.

12 Get a shell inside a container

Type this

```
$ docker run -it alpine sh
```

You should see

```
/ #
```

Your prompt changed. You are now **inside** the container. Try some commands.

Type this (inside the container)

```
/ # ls
bin    dev    home   media  opt    root   sbin   sys    usr
boot  etc    lib    mnt    proc   run    srv    tmp    var

/ # cat /etc/os-release
NAME="Alpine Linux"
VERSION_ID=3.20.3

/ # whoami
root
```

You are inside a small Linux system, logged in as the administrator. Leave it by typing **exit**.

Type this

```
/ # exit
$
```

Your normal prompt returns. The container has stopped, because the shell you were using *was* the container's job. When the job ends, the container ends.

Option	What it means
-i	Keep the connection open so you can type into the container.
-t	Give a proper terminal so text is laid out correctly.
-it	Both together. Always write it this way.

13 Enter a container that is already running

docker run always makes a **new** container. To step inside one that is already running, use **docker exec**. Start the web server again first.

Type this

```
$ docker run -d -p 8080:80 --name web nginx:alpine
$ docker exec -it web sh
```

You are inside the running web server. Look at the page it is serving.

Type this (inside the container)

```
/ # ls /usr/share/nginx/html
50x.html    index.html

/ # exit
```

This time the container **keeps running** after you exit, because the shell was an extra visitor, not the container's main job.

Type this

```
$ docker ps
```

CHECK YOUR WORK

The container **web** should still be listed as **Up**. Compare this with Step 12, where exiting stopped the container.

WARNING

Small images use **sh**, not **bash**. If you type `docker exec -it web bash` you will get **executable file not found**. Use **sh** and it works.

14 Prove that changes inside a container disappear

This short experiment teaches something important. Do it carefully.

Type this

```
$ docker exec -it web sh
/ # echo "<h1>I changed this page</h1>" > /usr/share/nginx/html/index.html
/ # exit
```

Refresh <http://localhost:8080>. Your new page appears.

Now delete the container and start a fresh one from the same image.

Type this

```
$ docker rm -f web
$ docker run -d -p 8080:80 --name web nginx:alpine
```

Refresh the browser again.

CHECK YOUR WORK

The **Welcome to nginx!** page is back. Your change is gone permanently. Anything you change inside a container is lost when that container is deleted.

There are two ways to make changes last, and you will learn both:

- To change the **program itself**, build your own image — Part E.
- To keep **information the program saves**, use a volume — Part F.

Part E: Building Your Own Image

So far you have used images made by other people. Now you will make your own.

15 Create the files

Make a new folder inside **docker-lab** and move into it.

Type this

```
$ mkdir mysite
$ cd mysite
```

Using your text editor, create a file called **index.html** in this folder with exactly this content:

FILE: INDEX.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Docker Page</title>
</head>
<body>
  <h1>Hello from inside a container</h1>
  <p>My name is [write your name here]</p>
</body>
</html>
```

Now create a second file in the same folder. It must be called **Dockerfile** — capital D, and **no file extension at all**.

FILE: DOCKERFILE

```
# Start from the small Nginx image
FROM nginx:alpine

# Copy my page into the folder Nginx serves
COPY index.html /usr/share/nginx/html/index.html

# Note which port this program uses
EXPOSE 80
```

WARNING

Windows Notepad secretly adds **.txt** to the filename, and then Docker cannot find the file. Use **Visual Studio Code** instead. If you must use Notepad, choose **All Files** in the *Save as type* box and type the name in quotes: **"Dockerfile"**.

Type this

```
$ ls
```

CHECK YOUR WORK

You should see exactly two files: **Dockerfile** and **index.html**. If you see **Dockerfile.txt**, rename it before continuing.

16 Build the image

Type this

```
$ docker build -t mysite:1.0 .
```

You should see

```
[+] Building 2.3s (7/7) FINISHED
=> [1/2] FROM docker.io/library/nginx:alpine
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => naming to docker.io/library/mysite:1.0
```

Two parts of that command need explaining.

The part **-t mysite:1.0** gives your image a **name** (**mysite**) and a **version** (**1.0**).

The dot at the end tells Docker which folder to use. It means *this folder*. It is a separate part of the command, not part of the name.

WARNING

Forgetting the dot is the most common mistake here. If you see `"docker build" requires exactly 1 argument`, you left the dot out.

What the dot really means

Think of Docker as a baker in another room. He cannot see your folder, so you must put everything he needs into a box and pass it to him. The dot says *"put this folder in the box"*.

This is why `COPY` can only use files from your folder. Anything outside was never put in the box, so the baker never receives it.

17 Run your own image

Type this

```
$ docker rm -f web
$ docker run -d -p 8080:80 --name mysite mysite:1.0
```

Visit <http://localhost:8080> again.

CHECK YOUR WORK

You should see **your own page**, with your name on it. The page is now part of the image, so any container you start from `mysite:1.0` will show it — on any computer, with no editing needed.

18 Build a Python application image

Now something more realistic. Go back up one folder and make a new one.

Type this

```
$ cd ..
$ mkdir pyapp
$ cd pyapp
```

Create three files in this folder.

```
FILE: APP.PY
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>My Python app is running in Docker</h1>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

WARNING

The part that says `host="0.0.0.0"` is essential. Without it, the program only listens to itself and your browser will never reach it, no matter what ports you use. Every server you put in a container needs this.

FILE: REQUIREMENTS.TXT

```
flask==3.0.3
```

FILE: DOCKERFILE

```
FROM python:3.12-alpine
```

```
# Work inside a folder called /app
```

```
WORKDIR /app
```

```
# Copy the list of needed libraries first, on its own
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Then copy everything else
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Type this

```
$ docker build -t pyapp:1.0 .
```

```
$ docker run -d -p 5000:5000 --name pyapp pyapp:1.0
```

```
$ docker logs pyapp
```

You should see

```
* Running on all addresses (0.0.0.0)
```

CHECK YOUR WORK

Visit <http://localhost:5000> in your browser. You should see your Python application. If the page will not load, check that `app.py` contains `host="0.0.0.0"`.

Understanding the new instructions

Instruction	What it does	When it happens
FROM	Choose the image to start from. Always first.	Building
WORKDIR	Make a folder and work inside it from now on.	Building
COPY	Copy files from your folder into the image.	Building
RUN	Run a command while building, such as installing libraries.	Building
EXPOSE	Write down which port the program uses. Does not open it.	Building
CMD	The command to run when a container starts .	Starting

NOTE

RUN and **CMD** look similar but are completely different. **RUN** happens once, while building, and its result is saved into the image. **CMD** does not run during the build at all — it waits until someone starts a container.

19 See why the file order matters

Open `app.py`, change the message inside the quotes, save it, and build again.

Type this

```
$ docker build -t pyapp:2.0 .
```

It finishes much faster than the first build. Look at the output and you will see the word **CACHED** next to some steps.

Docker remembers each instruction. When you build again, it reuses every step whose files have not changed, and only redoes the steps after the first change.

WHAT DOCKER REUSES

FROM python:3.12-alpine	CACHED	
WORKDIR /app	CACHED	
COPY requirements.txt .	CACHED	<- unchanged
RUN pip install ...	CACHED	<- saved 30 seconds
COPY . .	redone	<- app.py changed
CMD ["python", "app.py"]	redone	

This is why `requirements.txt` was copied on its own line, before the rest. If everything were copied together, changing one word in `app.py` would force all the libraries to install again, every single time.

TIP

The rule is simple: put things that rarely change at the **top** of the Dockerfile, and things that change often at the **bottom**.

Part F: Keeping Data Safe

In Step 14 you saw that changes vanish when a container is deleted. A **volume** is storage that Docker keeps separately, so it survives.

20 Create and use a volume

Type this

```
$ docker volume create mydata
$ docker volume ls
```

You should see

DRIVER	VOLUME NAME
local	mydata

Now attach it to a container. The `-v` option connects the volume to a folder inside the container.

Type this

```
$ docker run -it --name box1 -v mydata:/data alpine sh
/ # echo "This will survive" > /data/notes.txt
/ # cat /data/notes.txt
This will survive
/ # exit
```

Now delete that container completely.

Type this

```
$ docker rm box1
```

Start a brand new container and attach the same volume.

Type this

```
$ docker run -it --name box2 -v mydata:/data alpine sh
/ # cat /data/notes.txt
```

CHECK YOUR WORK

You should see **This will survive**. The first container was destroyed, but the file remained. This is how databases keep their information.

Type this

```
/ # exit
$ docker rm box2
```

Part G: Connecting Containers

Applications usually need more than one container — a program and a database, for example. They need to find each other.

21 Create a network

Type this

```
$ docker network create mynet
$ docker network ls
```

Now start two containers on that network.

Type this

```
$ docker rm -f mysite
$ docker run -d --name server --network mynet nginx:alpine
$ docker run -it --name client --network mynet alpine sh
```

You are inside the second container. Try to reach the first one **by its name**.

Type this (inside the container)

```
/ # ping -c 2 server
PING server (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.089 ms
```

```
/ # wget -qO- http://server
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

CHECK YOUR WORK

The name **server** worked as an address. Containers on the same network find each other by name automatically. This is how a program finds its database.

Type this

```
/ # exit
$ docker rm -f server client
```

NOTE

Notice that you did not need **-p** for this. **-p** is only needed to reach a container from **outside** Docker, such as from your browser. Containers talking to each other need nothing.

Part H: Running Several Containers Together

Typing long commands for every container is slow and easy to get wrong. **Docker Compose** lets you describe everything in one file and start it all with one command.

22 Build a visitor counter

This application has two containers: a Python program, and a Redis database that stores the count.

Type this

```
$ cd ..
$ mkdir counter
$ cd counter
```

Create four files in this folder.

FILE: APP.PY

```
from flask import Flask
import redis

app = Flask(__name__)

# "cache" is the service name from compose.yaml
store = redis.Redis(host="cache", port=6379)

@app.route("/")
def count():
```

```

    hits = store.incr("hits")
    return f"<h1>This page has been visited {hits} times</h1>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

FILE: REQUIREMENTS.TXT

```

flask==3.0.3
redis==5.0.8

```

FILE: DOCKERFILE

```

FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

```

FILE: COMPOSE.YAML

```

services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - cache

  cache:
    image: redis:7-alpine
    volumes:
      - cachedata:/data

volumes:
  cachedata:

```

Reading the compose file

- **services** lists the containers. The names **web** and **cache** become addresses — this is why **app.py** connects to **cache**.
- **build: .** means build this one from the Dockerfile here. **image:** means download a ready-made one.
- **ports** does the same job as **-p**.
- **depends_on** makes **cache** start first.
- **volumes** keeps the count even after the containers are removed.

NOTE

You did not create a network anywhere in this file. Compose makes one automatically and puts both services on it. That is why they can find each other by name.

23 Start everything with one command

Type this

```
$ docker compose up -d --build
```

You should see

```
[+] Running 3/3
Network counter_default    Created
Container counter-cache-1  Started
Container counter-web-1    Started
```

Type this

```
$ docker compose ps
```

CHECK YOUR WORK

Visit <http://localhost:5000> and refresh the page several times. The number should go up each time. The count is stored in the Redis container, not the Python one.

WARNING

Type `docker compose` with a **space**. Older guides on the internet show `docker-compose` with a hyphen — that is the old version and should not be used.

24 Stop everything

Type this

```
# Stop and remove the containers, but keep the saved count
$ docker compose down

# Start again – the count carries on from where it stopped
$ docker compose up -d
```

To delete the saved data as well, add `-v`.

Type this

```
$ docker compose down -v
```

CHECK YOUR WORK

Start it once more with `docker compose up -d` and visit the page. The counter has returned to 1, because the volume holding the count was deleted.

Part I: Cleaning Up

25 See how much space Docker is using

Type this

```
$ docker system df
```

You should see

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	7	2	286.4MB	198.1MB (69%)
Containers	5	1	24.3MB	18.7MB (76%)
Local Volumes	2	1	12.8MB	8.4MB (65%)
Build Cache	14	0	94.2MB	94.2MB

Because you used small images throughout, these numbers are modest. With full-sized images they would be several gigabytes.

26 Remove what you no longer need

Type this

```
# Remove all stopped containers
$ docker container prune

# Remove images no container is using
$ docker image prune -a

# Remove leftover build files
$ docker builder prune
```

Each command asks you to confirm. Type **y** and press Enter.

WARNING

Be careful with `docker volume prune`. It permanently deletes saved data, and it cannot be undone. Only use it when you are sure you no longer need anything stored.

Quick Reference

Containers

Command	What it does
<code>docker run <image></code>	Create and start a container.
<code>docker run -d ...</code>	Run in the background.
<code>docker run -it <image> sh</code>	Run and open a shell inside.
<code>docker run -p 8080:80 ...</code>	Open port 8080 on your computer.
<code>docker run --name web ...</code>	Choose a name.
<code>docker ps</code>	Show running containers.
<code>docker ps -a</code>	Show all containers, including stopped.
<code>docker stop <name></code>	Stop a container.
<code>docker start <name></code>	Start a stopped container.
<code>docker rm -f <name></code>	Stop and delete a container.

Command	What it does
<code>docker logs -f <name></code>	Watch a container's messages.
<code>docker exec -it <name> sh</code>	Open a shell in a running container.

Images

Command	What it does
<code>docker images</code>	List your images.
<code>docker pull <image></code>	Download an image.
<code>docker build -t name:1.0 .</code>	Build an image from the Dockerfile here.
<code>docker rmi <image></code>	Delete an image.

Volumes, networks, and Compose

Command	What it does
<code>docker volume create <name></code>	Make a volume.
<code>docker run -v name:/data ...</code>	Attach a volume.
<code>docker network create <name></code>	Make a network.
<code>docker run --network <net> ...</code>	Put a container on a network.
<code>docker compose up -d</code>	Start everything in the compose file.
<code>docker compose down</code>	Stop and remove everything.
<code>docker compose logs -f</code>	Watch messages from all services.

If Something Goes Wrong

What you see	What to do
Cannot connect to the Docker daemon	Docker Desktop is not running. Open it and wait for the whale icon to stop moving.
port is already allocated	Another program is using that port. Use a different left-hand number, such as <code>-p 8081:80</code> .
The container name is already in use	A container with that name still exists. Run <code>docker rm -f <name></code> and try again.
executable file not found: bash	Small images have no <code>bash</code> . Use <code>sh</code> instead.
"docker build" requires exactly 1 argument	You left out the dot at the end of the build command.

What you see	What to do
Dockerfile: no such file or directory	You are in the wrong folder, or the file was saved as <code>Dockerfile.txt</code> .
COPY failed: file not found	The file is not in the same folder as the Dockerfile, or the name is spelled differently.
The container stops immediately	Run <code>docker logs <name></code> to see the error message.
The browser page will not load	Check you used <code>-p</code> . If it is your own program, check it uses <code>host="0.0.0.0"</code> .
no space left on device	Run <code>docker system df</code> , then <code>docker image prune -a</code> .

Check What You Have Learned

Tick each item you can do without looking back at this guide.

- Explain the difference between an image and a container.
- Download an image and run it as a container.
- Run a container in the background and open it in a browser.
- Explain what the two numbers in `-p 8080:80` mean.
- Read a container's log messages.
- Stop, start, and delete a container.
- Open a shell inside a running container.
- Explain why changes made inside a container disappear.
- Write a Dockerfile and build an image from it.
- Explain why the dot at the end of `docker build` is needed.
- Explain why `requirements.txt` is copied before the rest of the code.
- Use a volume to keep data after a container is deleted.
- Put two containers on a network and make them talk by name.
- Write a `compose.yaml` and start it with one command.
- Free up disk space used by Docker.

TIP

Any item you cannot do is a signal to go back to that part of the guide and work through the steps again. Repeat the commands until they feel familiar — that is how they stay in your memory.